大模型数学与代数推理能力提升方法综合报告

1. 引言

大语言模型（Large Language Models, LLMs）在自然语言理解、问答和代码生成等任务上已经取得显著进展，但在数学推理，尤其是代数推理上，仍然存在稳定性不足、可验证性不强和泛化脆弱等问题[1]。与一般文本任务不同，数学与代数推理不仅要求模型理解自然语言题意，还要求其在多步推导过程中持续维护变量关系、遵守约束条件、完成精确计算，并在必要时能够对中间步骤进行检验和修正[1][2]。这意味着，数学推理不是单纯的“语言生成”问题，而是语言理解、结构化表示、符号操作、搜索与验证共同参与的复合型任务。

代数任务之所以比一般算术题更难，关键在于它依赖抽象变量与关系表达。模型不仅要从题面中识别数量、实体和条件，还要把这些信息转化为方程、函数、逻辑约束或程序片段，并在未知量尚未被赋值的情况下持续保持关系一致性[1][11]。如果模型只是依赖表面模式匹配，就很容易在推理早期引入隐性错误，并在后续步骤中不断放大，最终表现为“推理看似合理但结果错误”的现象[12][24]。

围绕这些问题，过去数年中形成了几条相对清晰的提升路径。第一类方法通过提示工程与解码策略优化，激发模型已有的推理潜能；第二类方法通过持续预训练、监督微调、数据合成和蒸馏等训练手段，把数学能力更深地写入模型参数；第三类方法通过程序执行、符号求解、自动形式化和神经符号混合架构，把模型擅长的语义理解与外部系统擅长的精确计算结合起来；第四类方法则在强化学习、过程奖励模型和测试时搜索层面上继续扩展推理深度与稳定性[2][3][5][7][8][19][21][22][23]。

本文围绕“大模型数学与代数推理能力如何提升”这一核心问题，对当前主流方法进行系统梳理，并重点回答四个子问题：一是数学与代数推理的核心困难是什么；二是相关技术路线经历了怎样的演进；三是对于相对较弱的大模型，Cheatsheet 与模型蒸馏分别适合什么场景；四是在避免大规模训练或重微调的前提下，还存在哪些效果较好、可工程落地的增强方案。全文力图从原理、效果、局限性和应用建议四个层面，形成一篇可独立阅读和使用的研究报告。

2. 数学与代数推理的核心挑战

2.1 数学推理不是普通文本生成任务

数学推理任务通常要求模型完成以下几个环节：题意解析、信息抽取、关系建模、步骤展开、计算或符号变换、答案验证以及结果表达。任何一个环节出现偏差，都可能导致最终失败[1]。例如，在数学应用题中，模型需要先将自然语言中的叙述性信息转化为数量关系；在代数题中，模型还必须识别未知量、构造方程组，并在未求得具体数值之前对变量关系进行稳定维护[11][24]。

与普通问答不同，数学推理对“中间状态”的正确性高度敏感。自然语言生成任务允许一定程度的近似表达，而数学推理中的一个符号错误、一个系数遗漏或一次不合法变形，都足以彻底改变答案[2][12]。因此，数学任务不是“只要最后说得通就行”，而是要求从中间步骤到最终答案都尽量处于可验证、可追踪的状态。

2.2 代数推理的特殊困难

代数推理比单纯算术更具挑战，主要体现在四个方面。

第一，变量绑定与关系保持困难。代数问题通常涉及一个或多个未知量，模型需要在整条推理链中保持变量含义一致，并正确处理变量之间的等价、依赖和约束关系[1][24]。

第二，抽象符号操作难度高。很多代数推理不是直接计算，而是先进行移项、合并同类项、代入、消元、构造辅助变量或变换表达式。模型如果只依赖自然语言表面模式，往往会在这些关键操作上失稳[11][24]。

第三，长链条误差传播严重。代数问题常常需要多步推导，而每一步都依赖前一步的正确性。当模型中间某一步出现轻微偏差时，错误会沿着后续推理快速扩散[4][20][24]。

第四，结果验证难度高。相比简单算术题只需核对数值结果，代数题常常需要验证方程是否满足条件、解是否落在合法定义域中、证明过程是否遵守逻辑规则，这类验证往往需要借助外部工具或形式化系统[10][11]。

2.3 评估维度不能只看最终答案

传统数学基准通常以最终答案准确率为核心指标，如 GSM8K、MATH、MultiArith、SVAMP、AQuA 等[1][2][3]。这类指标直观、易比较，但只能回答“模型是否答对”，很难解释“模型是如何答对或答错的”。对于代数推理尤其如此，因为模型可能通过错误路径偶然得到正确答案，也可能在大部分步骤都正确的情况下因最后一处细节出错而被判定失败[19][20][21]。

因此，当前研究越来越强调过程级评价，包括中间步骤是否一致、是否保留了关键约束、是否出现逻辑跳步、是否能自我纠错、是否能在外部验证反馈下修正错误等[19][20][21][24]。从工程实践来看，如果目标是构建可靠的数学求解系统，那么单看最终准确率远远不够，必须同时关注过程可解释性、错误可定位性和推理稳定性。

3. 技术演进脉络：从提示工程到神经符号与测试时扩展

3.1 基础能力探索阶段：通用模型的数学潜力与局限

早期研究首先回答了一个基础问题：通用大模型到底有没有数学推理潜力。相关工作发现，GPT-3 一类模型在简单算术和短链推理上已经展现出一定能力，但在复杂的量化推理、大学水平科学问题和长链条数学题上表现明显不足[2][3]。这说明大模型中确实存在可被激活的推理潜能，但这些潜能在未经专门设计的条件下难以稳定释放。

Minerva 是这一阶段的重要代表。它基于通用大模型，在更高质量的科学与数学语料上继续训练，显著提升了模型在量化推理任务上的表现；在超过 200 道本科层次的科学问题上，模型能够正确回答接近三分之一的问题[2]。Minerva 的意义不只在于“性能提高了”，更在于它证明了数学能力的提升不能只依赖通用互联网语料，而必须引入更系统的技术内容与解题结构。

3.2 思维链革命：从“会答题”到“会展开步骤”

2022 年以后，思维链（Chain-of-Thought, CoT）成为数学推理研究的分水岭。Zero-shot CoT 证明，仅通过在提示中加入类似“Let’s think step by step”的简单引导，模型在多个推理任务上的表现就会显著上升[3]。例如，在 MultiArith 上，准确率可从 17.7% 提升到 78.7%[3]。这说明很多大模型并不是完全没有推理能力，而是缺少被正确激活和展开的机制。

随后，自一致性（Self-Consistency）进一步改进了这一思路。该方法不再依赖单条贪心推理路径，而是让模型生成多条不同的解题链，再通过一致性聚合选出最可靠的答案。相关实验显示，自一致性在 GSM8K、SVAMP 和 AQuA 等数据集上都能带来明显的绝对准确率提升[4]。这说明数学问题常常允许多条候选路径，而“多样化探索 + 汇总选择”比单次生成更接近稳定推理。

但是，CoT 也很快暴露出局限。它能够改善“步骤是否显式展开”的问题，却不能根本解决“中间计算是否精确”“符号变换是否合法”“变量关系是否持续一致”等更深层问题[5][24]。也正因为如此，研究重心开始从自然语言推理步骤本身，逐步转向更结构化的中间表征与外部执行机制。

3.3 专门化训练阶段：面向数学能力的数据与模型

在提示工程展示出上限之后，研究者开始更系统地改造训练过程。一条重要结论是：数学能力的提升不仅取决于是否加入数学数据，更取决于加入什么样的数据、在哪个训练阶段加入，以及这些数据是否包含高质量的“解题过程”而不仅仅是“最终答案”[8]。

相关研究表明，在持续预训练（Continued Pre-Training, CPT）阶段引入高质量问题求解数据，往往比只在监督微调（SFT）阶段加入复杂数学样本更有效[8]。原因在于，SFT 更容易让模型学会输出格式、任务风格和指令服从，而 CPT 更有机会把解题模式和结构性规律写入模型内部表征。围绕这一思路，数学领域出现了一批专门化模型与训练框架，如 WizardMath、MAMMOTH 和 Llemma 等[7][9][10]。

其中，Llemma 是开源数学模型的重要代表。该模型在 Proof-Pile-2 上继续预训练，在 GSM8K 和 MATH 等基准上取得显著提升，说明“高质量数学语料 + 数学友好模型基础”能够让开源模型在数学任务上逼近甚至部分超越通用闭源模型[7]。WizardMath 与 MAMMOTH 则从指令数据构造和混合训练范式入手，进一步证明了数学任务并非只能通过更大模型来解决，更关键的是要构造面向数学结构的训练信号[9][10]。

3.4 神经符号与形式化路线：让模型负责理解，让工具负责求解

当研究进入代数与更高阶数学问题时，单纯依赖模型内部语言推理的局限变得更加明显。于是，程序辅助、符号求解与自动形式化逐渐成为主流方向。PAL 是一个代表性转折点。它让模型把题目翻译为可执行程序，由 Python 解释器完成计算，从而将“理解题目”与“准确计算”拆分给不同模块，在 GSM8K 上取得了超过使用 CoT 的 PaLM-540B 的成绩[5]。

不过，程序执行路线在代数问题上并不是万能的。原因在于，过程式代码更适合数值计算，但不天然适合处理尚未赋值的抽象未知变量。代数问题的核心往往在于“在未知量状态下保持关系结构”，而不是立即给出数值[11][24]。因此，后续工作逐渐把注意力转向更适合作为代数中间表示的方程、符号表达与形式化语言。

Math2Sym 展示了“LLM 负责语义解析，符号求解器负责严格求解”的有效性。模型先把数学应用题转成符号形式，再由外部系统进行求解，这一路线在少样本场景中已经表现出对传统纯语言模型的优势[11]。Autoformalization 则进一步把自然语言问题转成可由 Isabelle 等系统验证的形式化语句，不仅能提升结果可靠性，还能对推理过程提供外部一致性检验[12]。在更高阶的数学证明任务中，LLM 与符号系统的协同也被证明能够显著提高搜索效率与证明质量[13]。

3.5 过程监督、强化学习与测试时搜索

随着模型能力上升，仅靠静态训练数据已不足以支撑更复杂的数学推理。强化学习因而重新受到重视。对数学任务而言，只依据最终答案进行奖励的结果奖励模型（ORM）存在严重的信用分配问题，因为它无法区分“前面都对、最后一步错”和“中间全错但碰巧答对”这两种轨迹[19][20][21]。因此，研究逐渐转向过程奖励模型（PRM）和自动过程验证器，用更细粒度的反馈提升中间步骤质量[20][21]。

与此同时，测试时计算扩展（Inference-time Compute Scaling）成为另一条重要路径。其基本思想是：在不修改模型权重的前提下，通过让模型生成更多候选路径、展开更多分支或执行更强的搜索，来换取更高的推理质量[22][23]。自一致性可被视为最基础的测试时扩展方法；更进一步的 rStar-Math 与 ReST-MCTS* 则通过树搜索、价值评估与动态剪枝，把有限计算预算集中到最有希望的推理轨迹上，从而显著改善小模型和弱模型在复杂数学问题上的表现[22][23]。

4. 主流技术方法分类与机理分析

4.1 提示工程与解码策略

提示工程仍然是数学推理增强的最低门槛方案，主要包括 Zero-shot CoT、Few-shot ICL、自一致性、验证导向提示和多轮反思提示等[3][4][27]。这类方法的核心不在于教模型新知识，而在于通过更好的任务表述方式，促使模型把已有知识更有序地调动出来。

其优势在于成本低、见效快、无需训练，适合作为任何数学系统的默认起点。其局限在于，当题目需要精确计算、稳定符号操作或严格约束维护时，提示工程很容易碰到上限，因为它并未改变模型底层的计算与表示能力[1][5][24]。

4.2 专门化训练、数据合成与课程设计

专门化训练方法主要通过数学语料继续预训练、监督微调、课程学习和合成解题数据来提升能力[7][8][9][10]。这一路线的优势是可以显著提高模型对数学任务的整体适应性，并把某些解题模式“固化”到参数中；缺点则是训练成本高、更新慢，而且一旦训练目标设计不当，模型可能学到的是数学问题的表面格式，而不是深层逻辑结构[8][16][18]。

在这一类方法中，数据质量比数据规模更加关键。高质量的解题过程数据、错误修正轨迹和结构化中间步骤，通常比单纯增加题量更有价值[8]。此外，课程式训练与教师模型辅助数据合成，也越来越多地被用于为模型构造由浅入深的推理能力成长路径[8][16]。

4.3 程序辅助、工具增强与神经符号方法

程序辅助方法的核心思想是把“自然语言理解”和“精确执行”分开处理。PAL 让模型负责生成程序，Python 负责执行；MathPrompter 则通过生成多个表达式或程序片段来增强交叉验证；Math2Sym、Autoformalization 和符号求解器路线，则进一步把模型输出映射为方程、逻辑语句或形式化表达，由外部系统完成严格运算和验证[5][6][11][12]。

这类方法对代数任务尤其重要，因为代数错误往往集中在方程构造、符号转换、非法变形和约束遗漏上，而这些恰恰是符号系统最擅长发现和纠正的问题[11][12][13][24]。从工程角度看，神经符号方法通常比单纯堆叠提示更可靠，也更容易形成可解释、可验证的推理闭环。

4.4 强化学习、过程验证与自我纠错

强化学习方法主要通过奖励设计来改善模型在复杂推理空间中的探索行为。结果奖励模型适合做粗粒度优化，但在数学任务上信息过于稀疏，因此越来越多工作转向 PRM、自动过程验证器、步骤级纠错框架和过程偏好建模[19][20][21][24]。

此外，自我纠错也从简单的“请检查你的答案”发展为更结构化的“验证-修正”框架。相关研究表明，单纯依赖模型用同一套参数去检查自己往往效果有限，而当外部验证器、符号求解器或过程监督信号被引入后，模型的纠错能力会显著提高[12][20][24]。这说明，数学推理中的纠错不能只靠“自省”，更需要来自外部系统的结构化反馈。

4.5 检索增强与外部记忆

检索增强生成（RAG）在数学领域的价值，不只是补充事实知识，更在于补充规则、公式、解题范式和证明技巧[25][26]。标准文本 RAG 在数学场景中常常失效，原因是数学公式具有强结构性，普通分块与向量检索容易破坏表达式内部依赖关系[25]。因此，更有效的做法通常是结构化检索、公式感知检索、例题检索和过程片段检索。

在定理证明与高阶数学任务中，检索“结论”通常不如检索“证明过程”和“推导技术”有效。Mimetic Theorem Prover 一类工作表明，模型如果能获得足够相关的证明文本上下文，就能在一定程度上模仿并迁移复杂的证明策略[26]。对于代数问题而言，外部记忆和检索同样重要，因为很多错误并非来自不会算，而是来自没有及时调用正确公式、模板或验证规则。

5. Cheatsheet 方法的系统评估

5.1 方法定义与基本实现

Cheatsheet 方法的核心思想，是把大量演示示例中的知识压缩成一份简洁、可读、可重用的文本摘要，并在推理时把这份摘要作为上下文提供给模型[14]。这份 Cheatsheet 通常包含任务规则、常见模式、关键公式、判别提示、步骤模板和易错点提醒等内容，其作用类似于“高度压缩但面向求解的任务知识卡片”。

Cheatsheet 的一般实现可分为两个步骤。第一步是知识提炼，即向高能力模型提供多组演示样本，并要求其抽取对目标任务最关键的结构化知识；第二步是在实际推理时，只向目标模型提供这份摘要和少量格式示例，而不再提供完整演示集[14]。这种方式本质上是一种“显式知识搬运”，它不改变模型参数，却把原本分散在大量例题中的规律重新打包成对当前任务最有帮助的提示上下文。

5.2 为什么对弱模型特别有效

Cheatsheet 对弱模型有效，原因至少有三点。

第一，它减轻了模型对长上下文示例的依赖。相比 many-shot ICL 直接喂给大量示例，Cheatsheet 通过压缩后的规则摘要保留关键知识，避免小模型在长上下文中出现注意力分散和模式漂移[14]。

第二，它降低了从参数中临时检索知识的难度。对较弱模型来说，最困难的往往不是读懂一句话，而是在解题过程中持续调出正确规则并保持使用一致性。Cheatsheet 通过把规则显式放到眼前，起到了能

力补偿作用[14][15]。

第三，它具有较强的可解释性与可编辑性。研究表明，人工对 Cheatsheet 进行局部修改，可以直接纠正模型在特定任务中的偏差，例如在 Disambiguation QA 等任务上，人工修订后的 Cheatsheet 可以带来额外性能提升[14]。这一点对教育、研究和竞赛调试尤其重要，因为团队可以看见模型“参考了什么”，而不是面对一组不可见权重盲目调参。

5.3 效果、优势与局限

从实验结果看，Cheatsheet ICL 在多个推理任务中能够以不高于传统 few-shot ICL 的 token 预算取得更好表现，并在不少场景下接近甚至超越 many-shot ICL[14]。其主要优势包括：

第一，效率高。它通过压缩示例知识减少了输入长度与检索复杂度。

第二，可解释。规则、模板和启发式是显式文本，人可以直接读懂和修改。

第三，部署灵活。无需训练，尤其适合黑盒 API 与算力有限场景。

第四，对弱模型友好。小模型通常更能受益于清晰、结构化、任务特定的外部规则。

但 Cheatsheet 也有局限。它本质上是一种知识压缩，因此不可避免地存在信息损失。如果摘要提炼得不充分，模型可能重新退回到原有的错误先验；如果任务变化过快，Cheatsheet 需要重新设计；如果目标任务高度依赖复杂结构推导，单纯的文本摘要仍可能不如符号求解器或树搜索稳定[11][14][15]。

6. 模型蒸馏与 Cheatsheet 的比较

6.1 原理差异

模型蒸馏与 Cheatsheet 的根本差异在于知识存储位置不同。蒸馏试图把教师模型的能力压缩进学生模型参数中；Cheatsheet 则把知识保存在外部文本或记忆中，并在推理时动态注入[14][16]。前者是参数化知识迁移，后者是非参数化知识增强。

蒸馏通常需要教师模型生成大量高质量轨迹，再用这些轨迹对学生模型进行微调[16][17]。如果过程设计得好，学生模型可以在推理速度、部署成本和任务适配上得到收益。但这一路线对教师数据质量、学生模型容量和训练目标高度敏感，一旦轨迹噪声较大或学生容量不足，学生模型就可能只学到输出风格而不是深层规则[17][18]。

6.2 效果差异与典型风险

蒸馏的最大优势，是一旦训练完成，线上调用无需每次额外附加长提示，适合分布稳定、吞吐量高的生产环境[16][17]。其最大风险，则是“泛化与保真度悖论”：学生模型未必因为更忠实地模仿教师推理链就

一定表现更好，有时过度追求轨迹保真反而会削弱在新题型上的泛化[18]。这在代数任务中尤其明显，因为教师链条可能包含特定题型的解法习惯，而学生模型未必具备完整复现这些结构的能力。

相比之下，Cheatsheet 不追求把能力永久写入参数，而是通过显式知识提示来提升当前任务表现。它的风险主要在于上下文长度、知识覆盖率和摘要质量，而不是参数容量不足。因此，在任务变化频繁、需要快速迭代或调试的场景中，Cheatsheet 往往更稳妥[14][15]。

6.3 适用场景比较

维度	Cheatsheet / Dynamic Cheatsheet	模型蒸馏
知识位置	外部文本或外部记忆	模型参数
启动成本	低	高
更新速度	快，可在线调整	慢，通常需重新训练
可解释性	强	弱
线上时延	会增加部分上下文成本	训练后更低
对弱模型的友好度	高	受模型容量限制较大
对分布外变化的适应	较强	容易失配

综合来看，如果目标是快速增强弱模型、服务于研究探索、教育场景或变化频繁的题型，Cheatsheet 和 Dynamic Cheatsheet 更具优势[14][15]。如果目标是长期部署、调用规模大且训练预算充足，蒸馏仍然是可行路线，但应避免机械复制长 CoT，而更应蒸馏关键结构、判别信号与可泛化的步骤模板[16][17][18]。

7. 避免大规模训练或重微调的有效路径

7.1 严格不改权重的增强方案

如果完全不希望修改模型参数，那么当前最有效的方法通常不是单一技巧，而是几类推理期增强方案的组合。

第一类是提示与解码增强，包括 Zero-shot CoT、自一致性和验证导向提示。这类方法实现成本最低，适合作为所有系统的基础层[3][4]。

第二类是外部工具增强，包括程序执行、符号求解、形式化验证和步骤级纠错。这类方法对代数任务尤其有效，因为它能将高风险步骤外包给更可靠的外部系统[5][11][12][24]。

第三类是外部记忆与结构化检索，包括 Cheatsheet、Dynamic Cheatsheet、公式检索、例题检索和过程检索。这类方法能够在不训练模型的条件下，为其补充规则、模板和历史经验[14][15][25][26]。

第四类是测试时搜索，包括多路径采样、Best-of-N、自一致性、树搜索和过程价值评估。它们通过增加推理时的探索深度来提升模型上限，适合解决高难度、高价值问题[4][22][23]。

7.2 允许有限训练时的折中方案

如果允许有限但非大规模训练，那么更现实的折中方案通常包括三类。其一，是在持续预训练阶段加入更高质量的数学过程数据，而非简单堆积题目数量[8]。其二，是使用参数高效微调或小规模蒸馏，把数学任务中最稳定、最可复用的能力迁移到模型中[1][16][17]。其三，是把训练目标更多地放在错误检测、过程验证和自我修正上，而不仅仅是最终答案拟合[19][20][21][24]。

这类折中方案的意义在于，它们试图以较低成本逼近数学专门化训练的收益，同时保留一定的可迭代性。对于中等规模模型或竞赛型任务系统来说，这往往比“全量重训一个新模型”更可行。

7.3 推荐的工程组合路径

从实际构建数学或代数推理系统的角度看，较稳妥的路线通常是分层式组合，而不是押注某一个单点方法。

1. 先建立 `结构化提示 + 自一致性` 的基础推理层，获取低成本基线[3][4]。
2. 对高风险步骤接入 `Python / SymPy / 形式化验证器` ，把计算和验证从 LLM 中剥离[5][11][12]。
3. 为弱模型增加 `Cheatsheet / Dynamic Cheatsheet` ，把常用规则、公式和错误规避策略显式化[14][15]。
4. 对难题或高价值题型引入 `树搜索 / 过程价值评估 / 测试时扩展` ，提升上限[22][23]。
5. 如果任务稳定且调用量大，再考虑专门化训练或蒸馏，把收益较高的结构固化为模型能力[7][8][16]。

8. 综合分析与应用建议

8.1 不同方法的相对优势

从绝对性能看，数学专门化训练、神经符号方法、过程监督强化学习和测试时搜索通常代表更高的能力上限[7][8][19][22][23]。从成本收益比看，提示工程、Cheatsheet 和工具增强往往更适合快速落地[5][14][15]。从可解释性与可靠性看，外部工具、显式记忆和形式化验证明显优于“纯模型内部生成”路线[11][12][13]。

因此，方法选择不应只看排行榜，而应结合场景需求。如果系统重视可解释性与稳健性，应优先选择带外部验证环节的路线；如果系统追求高吞吐与低时延，可考虑在稳定任务上使用蒸馏后的模型；如果场

景以研究探索和题型快速迭代为主，则更适合使用 Cheatsheet、结构化检索和测试时搜索。

8.2 面向不同场景的建议

对于教育场景，应优先强调过程透明、错误可解释和步骤可验证，因此更适合采用 `Cheatsheet + 符号求解/验证` 的组合，并保留可供学生阅读的中间步骤[11][14][15]。

对于工业部署场景，如果题型相对稳定、请求量大且成本敏感，可以考虑“前期用结构化方法构造高质量轨迹，后期再做蒸馏或轻量微调”的路线，以兼顾性能和时延[16][17]。

对于竞赛和高难研究场景，建议采用更强的测试时扩展与搜索机制，并尽可能接入形式化验证或符号搜索模块，因为这类任务往往不是缺少知识，而是缺少足够深的推理探索和可靠的中间检查[13][22][23][26]。

对于弱模型或预算受限场景，最优先的通常不是训练，而是显式规则注入、外部工具与动态记忆。这类方案更容易获得稳定增益，也更便于定位问题和持续优化[14][15][25]。

8.3 未来发展趋势

未来数学与代数推理系统大概率会沿着四个方向继续演进。

第一，模型与工具的耦合会更紧密。未来系统不会只让模型“说出”答案，而会让模型在程序执行、符号计算、定理证明器和形式化验证器之间进行更智能的协同[11][12][13]。

第二，过程监督会继续强化。相比只优化最终结果，未来更可能把“中间步骤是否带来有效进展”作为核心训练与筛选信号[19][20][21]。

第三，推理期记忆会更加动态。Dynamic Cheatsheet 和更广义的外部记忆机制表明，模型在测试时学习和持续吸收局部经验是可行的，这种能力会越来越重要[15]。

第四，结构化检索与模仿式证明会继续发展。数学任务需要的不只是事实召回，更是与当前问题结构相匹配的推导经验、证明模板和形式化知识片段[25][26]。

9. 结论

大模型数学与代数推理能力的提升，已经从早期依赖提示工程的单点优化，发展为训练、推理、工具、验证、记忆与搜索共同作用的系统工程。对数学任务而言，单纯让模型“多说几步”固然有帮助，但真正决定上限的，仍然是中间表征是否合适、计算与验证是否可靠、错误是否可被及时发现与纠正[3][5][11][12][24]。

从方法谱系上看，CoT 与自一致性解决了“如何展开推理”的问题；专门化训练解决了“如何把数学能力更深地写入模型”的问题；程序辅助、符号求解与自动形式化解决了“如何把语言理解与精确推理分工协作”的问题；过程奖励模型、步骤级纠错和测试时搜索则进一步解决了“如何在复杂推理空间中提高稳定性和探索深度”的问题[4][7][8][19][20][21][22][23]。

对于弱模型而言，Cheatsheet 与 Dynamic Cheatsheet 提供了一条比重训练或盲目蒸馏更轻量、可解释、可快速迭代的路径；对于需要长期部署的稳定场景，蒸馏和专门化训练仍有价值，但前提是要尽量蒸馏结构化规律，而不是机械模仿长篇轨迹[14][15][16][17][18]。总体而言，一个高质量的代数推理系统不应把所有希望寄托在单一模型上，而应构建由大模型、外部知识、符号工具、验证模块和测试时搜索共同组成的闭环架构。只有在这样的体系下，大模型才有可能在复杂代数推导与高可靠数学任务中发挥稳定、可持续的作用。

参考文献

[1] A Survey on Large Language Models for Mathematical Reasoning. https://arxiv.org/pdf/2506.08446

[2] Solving Quantitative Reasoning Problems with Language Models. https://proceedings.neurips.cc/paper_files/paper/2022/file/18abbeef8cfe9203fdf9053c9c4fe191-Paper-Conference.pdf

[3] Large Language Models are Zero-Shot Reasoners. https://proceedings.neurips.cc/paper/2022/file/8bb0d291acd4acf06ef112099c16f326-Paper-Conference.pdf

[4] Self-Consistency Improves Chain of Thought Reasoning in Language Models. https://openreview.net/forum?id=1PL1NIMMrw

[5] PAL: Program-aided Language Models. https://proceedings.mlr.press/v202/gao23f/gao23f.pdf

[6] MathPrompter: Mathematical Reasoning using Large Language Models. https://aclanthology.org/anthology-files/anthology-files/anthology-files/pdf/acl/2023.acl-industry.4.pdf

[7] Llemma: An Open Language Model for Mathematics. https://arxiv.org/pdf/2310.10631

[8] Advancing Mathematical Reasoning in Language Models: The Impact of Problem-Solving Data, Data Synthesis Methods, and Training Stages. https://openreview.net/forum?id=GtpubstM1D

[9] WizardMath: Empowering Mathematical Reasoning for Large Language Models via Reinforced Evol-Instruct. https://arxiv.org/pdf/2308.09583

[10] MAMMOTH: Building Math Generalist Models through Hybrid Instruction Tuning. https://proceedings.iclr.cc/paper_files/paper/2024/file/b063829b922fdeb4fa3472dd3471ff43-Paper-Conference.pdf

[11] Math2Sym: A System for Solving Elementary Problems via Large Language Models and Symbolic Solvers. https://openreview.net/pdf?id=eQrkAPcGRF

[12] Don't Trust: Verify -- Grounding LLM Quantitative Reasoning with Autoformalization. https://arxiv.org/pdf/2403.18120

[13] Proving Olympiad Inequalities by Synergizing LLMs and Symbolic Reasoning. https://proceedings.iclr.cc/paper_files/paper/2025/file/06b35bf96174d97bae899d915a8b7210-Paper-Conference.pdf

[14] Distilling Many-Shot In-Context Learning into a Cheat Sheet. https://arxiv.org/pdf/2509.20820v1

[15] Dynamic Cheatsheet: Test-Time Learning with Adaptive Memory. https://arxiv.org/pdf/2504.07952

[16] Distilling Mathematical Reasoning Capabilities into Small Language Models. https://arxiv.org/html/2401.11864v5

[17] Capture the Key in Reasoning to Enhance CoT Distillation Generalization. https://aclanthology.org/2025.acl-long.21.pdf

[18] On the Generalization vs Fidelity Paradox in Knowledge Distillation. https://arxiv.org/pdf/2505.15442

[19] Reinforcement Learning for Reasoning in Small LLMs: What Works and What Doesn't. https://arxiv.org/html/2503.16219v1

[20] Rewarding Progress: Scaling Automated Process Verifiers for LLM Reasoning. https://openreview.net/forum?id=A6Y7AqlzLW

[21] R-PRM: Reasoning-Driven Process Reward Modeling. https://aclanthology.org/2025.emnlp-main.679.pdf

[22] rStar-Math: Small LLMs Can Master Math Reasoning with Self-Evolved Deep Thinking. https://arxiv.org/abs/2501.04519

[23] ReST-MCTS*: LLM Self-Training via Process Reward Guided Tree Search. https://arxiv.org/pdf/2406.03816

[24] Enhancing Mathematical Reasoning in LLMs by Stepwise Correction.
https://aclanthology.org/2025.acl-long.1048.pdf

[25] RAG+: Enhancing Retrieval-Augmented Generation with Application-Aware Reasoning.
https://preview.aclanthology.org/bulk-verify-all/2025.emnlp-main.1630.pdf

[26] Retrieval-Augmented Language Models are Mimetic Theorem Provers.
https://aclanthology.org/2025.findings-emnlp.1162/

[27] What Makes In-context Learning Effective for Mathematical Reasoning.
https://openreview.net/pdf?id=upSPbQoVil